Feature spec: Import CSV / Excel

Overview
Implement server and UI import features for these data types: pesée (weighing), inventaire, mouvement de caisse, mouvement de stock, and caisse records. Support both CSV and Excel uploads, provide preview and validation, and persist records with clear error reporting and logs.

Goals

- Allow users to upload CSV or Excel files for the listed import types.
- Show a preview of parsed rows and detected column mapping before applying.
- Validate rows and report per-row errors; allow the user to fix or skip invalid rows.
- Import data transactionally with progress and import logs.
- Provide automated unit and integration tests for parsers and services.

Features (per import type)

- Input formats: CSV (UTF-8, comma or semicolon) and Excel (.xls, .xlsx).
- Required fields: each import type must declare required columns (see examples below).
- Column mapping: UI to map file columns to entity fields; auto-detect common headers.
- Preview: show first N rows, validation status, and suggested mappings.
- Import endpoint: POST /api/import/{type} with file multipart upload and optional mapping JSON.
- Service: Import{Type}Service handles parsing, validation, mapping, and persistence.
- Transactional: rollback on fatal errors; support partial import with skip/continue option.
- Logging & audit: record import job metadata, user, timestamp, counts (imported, skipped, failed), and an error log file.

Example required columns

- Pesée (pesee): date, bovin_id (or tag), poids_kg, lieu, operateur
- Inventaire: date, produit_id (or ref), quantite, emplacement, type_operation
- Mouvement caisse: date, type (entrée/sortie), montant, reference, motif
- Mouvement stock: date, produit_id, quantite, type (in/out), reference
- Caisse (general): date, compte, montant, type, description

PROF

Validation rules

- Required fields must be present and non-empty.
- Dates parsed in ISO (yyyy-MM-dd) or common local formats; invalid dates flagged.
- Numeric fields must parse to numbers and respect logical constraints (e.g., poids > 0).
- Referential checks: if a referenced entity (bovin, produit, compte) is missing, flag as error and optionally offer "create missing" behavior.

API / Backend contract

- Endpoint: POST /api/import/{type}
  - Params: multipart file, mapping JSON (optional), options {skipInvalid: boolean, preview: boolean}
  - Response (preview): parsedRows[], detectedMapping, validationSummary

    - Response (apply): jobId, importedCount, skippedCount, failedCount, errorLogUrl

UI UX

- Single upload page with tabs for each import type.
- File chooser + drag-and-drop, mapping UI, preview table with validation column, and a final "Import" button.
- Show progress bar and link to import log when finished.

Persistence & Jobs

- Store import jobs in a table `import_job` with status, user, timestamps, counts, and link to error file.
- Write a small background worker or use synchronous processing for small files; stream large files to avoid OOM.

Testing

- Unit tests for parsers (CSV and Excel) and validators.
- Integration test for the full import flow using sample files in `test/resources`.

Acceptance criteria

- Uploading a valid CSV/XLSX for any supported type results in persisted records and a success job log.
- Invalid rows are reported and do not corrupt existing data.
- Preview and mapping UI is usable and suggests common header mappings.

Next steps (implementation roadmap)

1. Add API controller and endpoints for file upload and preview.
2. Implement parser utilities for CSV and Excel with configurable column mapping.
3. Implement `Import{Type}Service` for each domain type with validation and persistence.
4. Add UI views/pages for upload, mapping, preview and job status.
5. Add import job persistence and error logging.
6. Write unit and integration tests and sample fixture files.

PROF

Example CSV header for `pesee`:
date,bovin_tag,poids_kg,lieu,operateur

Notes

- Keep parsing tolerant: trim values, ignore empty rows, allow optional headers order.
- For ambiguous references (e.g., bovin by tag vs id), allow a mapping option in the UI.